AI Usage Report

Course: Software for Mobile Devices (SMD)
Assignment: Assignment 1 — Open-Ended, AI-Assisted Application Development (Reimagine FLEX Student Portal)
Student Name: Muhammad Abubakar Ali
Registration No.: 22i-2693
Date: 03-Oct-2026

1. AI Tool(s) Used

Prompt:
List the AI tools used during the development of your application.

Response:

- **Primary Tools:** ChatGPT, Google Gemini / Antigravity AI
- **Environment:** React Native (Expo SDK 57), Node.js, JavaScript (ES6+)

2. Purpose of AI Usage

Prompt:
Explain what you used AI for during the development of your application.

Response:

AI assistance was utilized across multiple stages of application development:

- **UI/UX Architecture & Theming:** Designing a modern, slate-dark dashboard aesthetic tailored for mobile screens with high visual hierarchy, micro-interactions, and accessible contrast.
- **Component Architecture:** Structuring modular components without forbidden external navigation dependencies (pure React state-based view switching between Dashboard, Courses, Tasks, and Analytics).
- **Data Visualization with react-native-chart-kit:** Setting up responsive BarChart (for course-wise attendance vs threshold) and PieChart (for workload breakdown) that update dynamically when data changes.
- **Mathematical & Projection Logic:** Formulating pure JavaScript functions for attendance calculations, safe division handling, dynamic warning generation, and "what-if" projection simulation.
- **Form Validation & Edge-Case Handling:** Implementing controlled form inputs with inline validation for adding tasks and testing empty data states.

3. Important Prompts Used

Prompt 1:

"How do I structure a multi-view React Native application with zero external navigation libraries (no React Navigation, no bottom tabs, no sidebars on mobile) using pure lifted state in App.js with clean conditional rendering?"

Prompt 2:

"Generate responsive charting code using react-native-chart-kit (BarChart and PieChart) in React Native that dynamically maps from course attendance data and task priority states, ensuring charts resize correctly on different screen widths and do not crash on empty data."

Prompt 3:

"Write a pure JavaScript calculation utility function that calculates attendance percentages, determines status (Healthy >= 85%, Watch 75-84%, Low < 75%), handles 0 total classes safely, and provides a 'what-if' projection function for hypothetical upcoming classes."

4. AI-Generated Output

Prompt:
Briefly describe the code, solution, or suggestion provided by the AI.

Response:

The AI provided:

1. A modular file layout (App.js, views/, components/, data/, utils/, constants/) with centralized theme tokens in theme.js.
2. Initial scaffolding for calculateAttendance, calculateOverallAttendance, getCoursesNeedingAttention, and projectAttendance in utils/calculations.js.
3. Configuration objects and dimensions wrappers for react-native-chart-kit BarChart and PieChart components.
4. Boilerplate for modal dialogs (CourseDetailModal.js and AddTaskModal.js) with React Native Modal, TextInput, and KeyboardAvoidingView.

5. Changes Made by Me

Prompt:
Explain what you changed, modified, or improved in the AI-generated solution.

Response:

- **Enforced Strict Class Constraints:** Removed experimental drawer/bottom bar snippets and ensured only class-approved React Native core components (View, Text, ScrollView, TouchableOpacity, Modal, TextInput, SafeAreaView) were used.
- **Interactive Attendance Simulation:** Connected the course card tap and modal buttons directly to lifted state in App.js (handleAttendClass, handleMissClass) so tapping immediately recalculates dashboard stats, updates the chart, and adjusts warning badges in real time.
- **Attendance Projection Simulator:** Expanded the course detail modal to include an interactive number stepper for testing "What will my attendance be if I attend/miss the next N classes?"
- **Responsive Chart Constraints:** Fixed static chart widths by computing Math.min(width - 64, 420) wrapped in a reusable ChartCard container to eliminate horizontal scroll glitches on physical devices.
- **Centralized Viva Thresholds:** Placed ATTENDANCE_WARNING_THRESHOLD = 75 and ATTENDANCE_HEALTHY_THRESHOLD = 85 into constants/theme.js for instant modification during the viva.

6. My Understanding

Prompt:
Explain the important part(s) of the AI-generated code in your own words.

Response:

- **Lifted State & Unidirectional Data Flow:** All primary data (courses, tasks, currentView, selectedCourseForDetail) lives in App.js. Child views receive this data as props and communicate changes back up using callback functions (onAttendClass, onToggleTask, onAddTask).
- **Derived State:** Metrics such as overall attendance, average marks, attention course count, and active warnings are not stored in separate useState hooks. Instead, they are calculated on every render pass directly from courses and tasks arrays using reduce(), filter(), and map(). This completely prevents out-of-sync state bugs.
- **Pure JavaScript Utilities:** utils/calculations.js consists of pure functions with zero side effects. For example, calculateAttendance(attended, total) returns 0 if total === 0, preventing NaN or division-by-zero crashes.
- **Conditional Rendering:** Views are swapped using standard JavaScript logical expressions ({currentView === 'dashboard' && <DashboardView ... />}) without any third-party router overhead.

7. Verification and Testing

Prompt:
Explain how you tested the AI-generated solution.

Response:

- **Testing Methods:** Tested on Android device via Expo Go, web browser (npm run web), and local Node syntax checks.
- **Errors Encountered & Fixes:**
    1. **NaN in Attendance:** When total classes were 0, the percentage calculated as NaN. Fixed by adding the guarded check if (!totalClasses || totalClasses <= 0) return 0;.
    2. **Chart Overflow on Small Screens:** The default chart width caused horizontal layout clipping. Fixed using dynamic window dimension calculations with safe margin clamps.
    3. **Pie Chart Crash on 0 Tasks:** If all tasks were cleared or completed, PieChart received empty datasets. Added empty data filtering and fallback UI rendering.
    4. **Stale Modal Data:** Modal attendance did not update live when tapping attend. Fixed by looking up the live course object from the latest courses array state.
- **Result:** All features (charts, attendance mutation, task filters, validation, what-if projection) function smoothly with zero runtime errors.

8. Reflection

Prompt:
What did you learn from using AI during this assignment?

Response:

Using AI highlighted the importance of being an active engineering director rather than a passive code consumer. AI excels at generating boilerplate, styling variants, and charting configurations rapidly. However, ensuring pedagogical constraints (such as strictly avoiding unapproved navigation packages, handling edge-case arithmetic, and establishing clean unidirectional data flow) required critical thinking, manual refactoring, and deep conceptual understanding of React lifecycle and JavaScript array operations.

Student Declaration

I confirm that I have used AI tools only as a development assistant and that I understand the code submitted as part of this assignment. I am able to explain and demonstrate the functionality of my application.

Student Name: Muhammad Abubakar Ali

Signature:



Date: 03-Oct-2026